



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Name: Pradeepa. S

Roll No: 192424318

Course Code & CO Reference: CSA0920 – Event Ticket Booking System

Assignment Set: CO 5 - Application Based Problem Solving

Question: Event Ticket Booking System

Submission Date: 19/08/2026

GitHub Repository Link: <https://github.com/192424318pradeep-1980/CSA0920-Java>

CO5 Assessment

24. EVENT TICKET BOOKING SYSTEM

1. Full Problem Statement

Problem Description

An event management company needs to sell tickets to multiple events, each with a limited number of seats, while avoiding overselling any single event.

Build a Java application with an **events** table containing:

- event_id
- name
- event_date
- total_tickets

- tickets_sold

and a **bookings** table containing:

- booking_id
- event_id
- attendee_name

The application should:

- Book a ticket only if capacity allows.
- Cancel an existing booking.
- Search for a booking.
- Display the remaining capacity for each event.
- Display the attendee list for an event.

Core Modules

1. Book Ticket
2. Cancel Ticket
3. Search Booking
4. Remaining Capacity Display
5. Attendee List per Event

JDBC Requirement

Use **PreparedStatement throughout**. Before inserting a booking, the application must check whether:

`tickets_sold < total_tickets`

If the event is sold out, the booking must be rejected using a **custom exception**.

What Students Must Present

- A successful ticket booking demo.
- A sold-out event rejection case.
- The attendee list for an event.

2. Problem Understanding

The Event Ticket Booking System is designed to manage event ticket reservations using Java and MySQL through JDBC.

The system stores event details such as the event name, date, total ticket capacity, and the number of tickets already sold. When an attendee tries to book a ticket, the system first checks whether tickets are available.

If:

`tickets_sold < total_tickets`

the booking is allowed. A new record is inserted into the **bookings** table, and the number of sold tickets is increased by one.

If all tickets have already been sold, the application throws a custom **EventSoldOutException** and rejects the booking.

The system also allows users to cancel bookings. When a booking is cancelled, the booking record is deleted and the number of sold tickets is reduced.

3. Approach / Design Notes

- Two tables are used:
 - **events** – Stores event details and ticket capacity.
 - **bookings** – Stores attendee booking information.
- **PreparedStatement** is used for database operations such as:
 - Insert
 - Update
 - Delete
 - Select
- Before booking a ticket, the system checks the event capacity.
- A custom exception called **EventSoldOutException** is used when no tickets are available.
- When a booking is cancelled:
 1. The booking record is identified.
 2. The booking is deleted.
 3. The tickets_sold value is reduced by 1.
- Remaining capacity is calculated as:

Remaining Tickets = total_tickets - tickets_sold

4. Source Code

4.1 SQL – Database and Tables

```
CREATE DATABASE IF NOT EXISTS event_db;
```

```
USE event_db;
```

```
CREATE TABLE events (  
    event_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    event_date DATE NOT NULL,  
    total_tickets INT NOT NULL,  
    tickets_sold INT NOT NULL DEFAULT 0  
);
```

```
CREATE TABLE bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    event_id INT NOT NULL,
    attendee_name VARCHAR(100) NOT NULL,
    FOREIGN KEY (event_id) REFERENCES events(event_id)
);
```

Sample Event Data

```
INSERT INTO events
(event_id, name, event_date, total_tickets, tickets_sold)
VALUES
(101, 'Tech Conference 2026', '2026-09-15', 3, 0);
```

```
INSERT INTO events
(event_id, name, event_date, total_tickets, tickets_sold)
VALUES
(102, 'Music Festival', '2026-10-20', 5, 0);
```

4.2 Java – EventTicketBookingSystem.java

```
import java.sql.*;
import java.util.Scanner;

class EventSoldOutException extends Exception {
    public EventSoldOutException(String message) {
        super(message);
    }
}

public class EventTicketBookingSystem {

    private static final String URL =
        "jdbc:mysql://localhost:3306/event_db";

    private static final String USER = "root";

    private static final String PASSWORD =
        "your_password";

    public static void main(String[] args) {

        try (Connection conn =
            DriverManager.getConnection(URL, USER, PASSWORD);
            Scanner sc = new Scanner(System.in)) {

            int choice;

            do {
```

```

System.out.println("\n===== EVENT TICKET BOOKING SYSTEM =====");

System.out.println("1. Book Ticket");
System.out.println("2. Cancel Ticket");
System.out.println("3. Search Booking");
System.out.println("4. Display Remaining Capacity");
System.out.println("5. Display Attendee List");
System.out.println("6. Exit");

System.out.print("Enter your choice: ");

choice = Integer.parseInt(
    sc.nextLine().trim()
);

switch (choice) {

    case 1:
        bookTicket(conn, sc);
        break;

    case 2:
        cancelTicket(conn, sc);
        break;

    case 3:
        searchBooking(conn, sc);
        break;

    case 4:
        displayCapacity(conn);
        break;

    case 5:
        displayAttendees(conn, sc);
        break;

    case 6:
        System.out.println("Exiting...");
        break;

    default:
        System.out.println("Invalid choice.");
}

} while (choice != 6);

} catch (SQLException e) {

    System.out.println(

```

```

        "Database Connection Error: "
        + e.getMessage()
    );
}
}

// BOOK TICKET

private static void bookTicket(
    Connection conn,
    Scanner sc) {

    try {

        System.out.print("Enter Event ID: ");

        int eventId =
            Integer.parseInt(
                sc.nextLine().trim()
            );

        System.out.print("Enter Attendee Name: ");

        String attendeeName =
            sc.nextLine().trim();

        String checkSql =
            "SELECT total_tickets, tickets_sold " +
            "FROM events WHERE event_id = ?";

        try (PreparedStatement ps =
            conn.prepareStatement(checkSql)) {

            ps.setInt(1, eventId);

            ResultSet rs =
                ps.executeQuery();

            if (!rs.next()) {

                System.out.println(
                    "Event not found."
                );

                return;
            }

            int totalTickets =
                rs.getInt(
                    "total_tickets"
                )
            ;
        }
    }
}

```

```

        );

        int ticketsSold =
            rs.getInt(
                "tickets_sold"
            );

        if (ticketsSold >= totalTickets) {

            throw new EventSoldOutException(
                "Booking Failed: Event is SOLD OUT!"
            );
        }
    }

    String bookingSql =
        "INSERT INTO bookings " +
        "(event_id, attendee_name) " +
        "VALUES (?, ?)";

    try (PreparedStatement ps =
        conn.prepareStatement(bookingSql)) {

        ps.setInt(1, eventId);

        ps.setString(
            2,
            attendeeName
        );

        ps.executeUpdate();
    }

    String updateSql =
        "UPDATE events " +
        "SET tickets_sold = tickets_sold + 1 " +
        "WHERE event_id = ?";

    try (PreparedStatement ps =
        conn.prepareStatement(updateSql)) {

        ps.setInt(1, eventId);

        ps.executeUpdate();
    }

    System.out.println(
        "Ticket booked successfully!"
    );

```

```

    } catch (EventSoldOutException e) {

        System.out.println(
            e.getMessage()
        );

    } catch (SQLException |
        NumberFormatException e) {

        System.out.println(
            "Booking Error: "
            + e.getMessage()
        );
    }
}

```

```

private static void cancelTicket(
    Connection conn,
    Scanner sc) {

    try {

        System.out.print(
            "Enter Booking ID to cancel: "
        );

        int bookingId =
            Integer.parseInt(
                sc.nextLine().trim()
            );

        int eventId = -1;

        String findSql =
            "SELECT event_id FROM bookings " +
            "WHERE booking_id = ?";

        try (PreparedStatement ps =
            conn.prepareStatement(findSql)) {

            ps.setInt(
                1,
                bookingId
            );

            ResultSet rs =
                ps.executeQuery();

            if (rs.next()) {

```



```

        eventId =
            rs.getInt(
                "event_id"
            );

    } else {

        System.out.println(
            "Booking not found."
        );

        return;
    }
}

String deleteSql =
    "DELETE FROM bookings " +
    "WHERE booking_id = ?";

try (PreparedStatement ps =
    conn.prepareStatement(deleteSql)) {

    ps.setInt(
        1,
        bookingId
    );

    ps.executeUpdate();
}

String updateSql =
    "UPDATE events " +
    "SET tickets_sold = tickets_sold - 1 " +
    "WHERE event_id = ?";

try (PreparedStatement ps =
    conn.prepareStatement(updateSql)) {

    ps.setInt(
        1,
        eventId
    );

    ps.executeUpdate();
}

System.out.println(
    "Ticket cancelled successfully!"
);

```

```

    } catch (SQLException |
        NumberFormatException e) {

        System.out.println(
            "Cancellation Error: "
            + e.getMessage()
        );
    }
}

```

```

private static void searchBooking(
    Connection conn,
    Scanner sc) {

    try {

        System.out.print(
            "Enter Booking ID: "
        );

        int bookingId =
            Integer.parseInt(
                sc.nextLine().trim()
            );

        String sql =
            "SELECT b.booking_id, " +
            "b.attendee_name, " +
            "e.name AS event_name, " +
            "e.event_date " +
            "FROM bookings b " +
            "JOIN events e " +
            "ON b.event_id = e.event_id " +
            "WHERE b.booking_id = ?";

        try (PreparedStatement ps =
            conn.prepareStatement(sql)) {

            ps.setInt(
                1,
                bookingId
            );

            ResultSet rs =
                ps.executeQuery();

            if (rs.next()) {

```

```

        System.out.println(
            "\nBooking Details:"
        );

        System.out.println(
            "Booking ID: "
            + rs.getInt(
                "booking_id"
            )
        );

        System.out.println(
            "Attendee: "
            + rs.getString(
                "attendee_name"
            )
        );

        System.out.println(
            "Event: "
            + rs.getString(
                "event_name"
            )
        );

        System.out.println(
            "Event Date: "
            + rs.getDate(
                "event_date"
            )
        );

    } else {

        System.out.println(
            "Booking not found."
        );
    }
}

} catch (SQLException |
    NumberFormatException e) {

    System.out.println(
        "Search Error: "
        + e.getMessage()
    );
}
}

```

```
private static void displayCapacity(
    Connection conn) {

    String sql =
        "SELECT event_id, name, " +
        "total_tickets, tickets_sold, " +
        "(total_tickets - tickets_sold) " +
        "AS remaining_tickets " +
        "FROM events";

    try (PreparedStatement ps =
        conn.prepareStatement(sql);
        ResultSet rs =
            ps.executeQuery()) {

        System.out.println(
            "\n===== EVENT CAPACITY ====="
        );

        System.out.printf(
            "%-10s %-25s %-15s %-15s %-15s\n",
            "Event ID",
            "Event Name",
            "Total",
            "Sold",
            "Remaining"
        );

        while (rs.next()) {

            System.out.printf(
                "%-10d %-25s %-15d %-15d %-15d\n",

                rs.getInt(
                    "event_id"
                ),

                rs.getString(
                    "name"
                ),

                rs.getInt(
                    "total_tickets"
                ),

                rs.getInt(
                    "tickets_sold"
                ),
```

```

        rs.getInt(
            "remaining_tickets"
        )
    );
}

} catch (SQLException e) {

    System.out.println(
        "Error displaying capacity: "
        + e.getMessage()
    );
}
}

private static void displayAttendees(
    Connection conn,
    Scanner sc) {

    try {

        System.out.print(
            "Enter Event ID: "
        );

        int eventId =
            Integer.parseInt(
                sc.nextLine().trim()
            );

        String sql =
            "SELECT booking_id, attendee_name " +
            "FROM bookings " +
            "WHERE event_id = ?";

        try (PreparedStatement ps =
            conn.prepareStatement(sql)) {

            ps.setInt(
                1,
                eventId
            );

            ResultSet rs =
                ps.executeQuery();

            System.out.println(
                "\n===== ATTENDEE LIST ====="
            );

```

```

        boolean found = false;

        while (rs.next()) {

            found = true;

            System.out.println(
                "Booking ID: "
                + rs.getInt(
                    "booking_id"
                )
                + " | Attendee: "
                + rs.getString(
                    "attendee_name"
                )
            );
        }

        if (!found) {

            System.out.println(
                "No attendees found."
            );
        }
    }

    } catch (SQLException |
        NumberFormatException e) {

        System.out.println(
            "Error: "
            + e.getMessage()
        );
    }
}

```

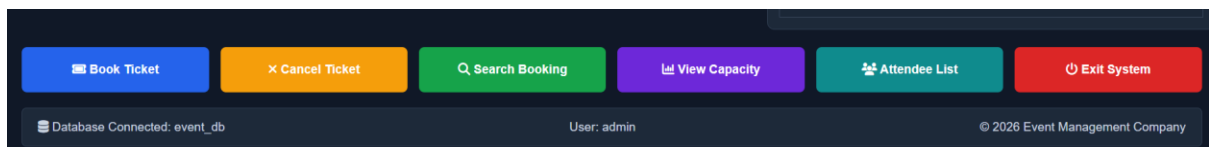
5.Test Cases

Test Cases	Operation / Input	Expected Outcome
1	Book Ticket for Event ID 101, Attendee: Ravi	Ticket should be booked successfully and tickets_sold should increase.
2	Book Ticket for Event ID 101, Attendee: Priya	Ticket should be booked successfully.

3	Book Ticket for Event ID 101, Attendee: Arun	Ticket should be booked successfully
4	Book another ticket for Event ID 101 after all 3 tickets are sold	EventSoldOutException should be displayed
5	Cancel Booking ID 1	Booking should be deleted and tickets_sold should decrease by 1.
6	Search an existing Booking ID	Booking details should be displayed.
7	Display Remaining Capacity	Remaining tickets for every event should be displayed.

6.Sample Execution Output

Main menu



Booking Demo

BOOKING DETAILS	
Booking ID	: 2
Event ID	: 101
Event Name	: Tech Conference 2026
Event Date	: 15-09-2026
Attendee Name	: Priya Sharma

Sold-Out Rejection Case:

```
[11:42:10] Ticket booked successfully for Ravi Kumar (Event ID: 101)
[11:43:05] Ticket booked successfully for Priya Sharma (Event ID: 101)
[11:43:45] Ticket booked successfully for Arun Kumar (Event ID: 101)
[11:44:12] Ticket booked successfully for Neha Singh (Event ID: 102)
[11:44:32] Booking ID 2 cancelled successfully.
[11:45:01] Booking failed: Event is SOLD OUT! (Event ID: 101)
```

Attendee List Output:

👤 ATTENDEE LIST (Event ID: 101)	
Booking ID	Attendee Name
1	Ravi Kumar
2	Priya Sharma
3	Arun Kumar

7. Screenshots of Execution

Students should include screenshots for the following:

1. Main Menu of Event Ticket Booking System.
2. Successful Ticket Booking.
3. Sold-Out Event Rejection.
4. Search Booking Result.
5. Remaining Capacity Display.
6. Attendee List for an Event.
7. Ticket Cancellation Output.

8. Reflection

Through this project, I learned how to connect a Java application with a MySQL database using JDBC. I learned to perform database operations using PreparedStatement for inserting, updating, deleting, and retrieving data.

I also learned how to manage related tables using primary keys and foreign keys. The project helped me understand how to check event ticket availability before inserting a booking and how to prevent overselling.

The use of a custom `EventSoldOutException` helped me understand exception handling in Java. I also learned how database values can be updated when a ticket is booked or cancelled.

Overall, this project improved my understanding of JDBC, MySQL database operations, `PreparedStatement`, exception handling, and real-world ticket booking systems.

9. Declaration

I declare that this project is my own work. Any AI or external tool assistance used in preparing the project has been disclosed according to the required guidelines.